

Agentic AI: Foundations and Open-Source Practice

Day 2 Lab Manual

Tracing ReAct on Paper, and Comparing Answers With and Without Chain-of-Thought

1. Lab Details

Lab Title	Trace a ReAct run on paper for a multi-step question; prompt an LLM with and without Chain-of-Thought and compare the answers
Unit	Unit 1: Foundations of AI Agents (sub-topics 1.3 and 1.4)
Editor	Visual Studio Code, using the day1_lab folder you created yesterday
LLM	The same option you used on Day 1: Ollama, Groq or Hugging Face
Duration	About 1.5 to 2 hours
Output	A completed paper ReAct trace, three new Python programs, and a completed comparison table

2. Aim

To trace the Thought, Action and Observation cycle of a ReAct agent by hand for a multi-step question, and then to measure the effect of Chain-of-Thought prompting by asking the same questions with and without step-by-step reasoning.

3. Objectives

After completing this lab, you will be able to:

1. Break a multi-step question into Thought, Action and Observation steps on paper.
2. Predict which tools an agent will call, and in what order, before running it.
3. Compare your paper trace with the trace an actual agent produces.
4. Write a direct prompt and a Chain-of-Thought prompt for the same question.
5. Measure how step-by-step reasoning changes the accuracy of a small model.
6. Apply self-consistency by sampling several reasoning paths and taking the majority answer.
7. Explain when Chain-of-Thought is enough and when tools (ReAct) are required.

4. Prerequisites

4.1 From Day 1

- Your day1_lab folder with .venv, .env, config.py, tools.py and agent.py working.
- check_setup.py must still print SETUP OK. Run it first if you are unsure.
- Ollama running (Option A), or a valid key in .env (Options B and C).

4.2 Knowledge

- Day 2 theory: the five components, Chain-of-Thought, and the ReAct cycle.
- Day 1 lab: how the agent printed its tool calls.

4.3 Materials for Part A

- A sheet of paper or the worksheet in Section 8.1, and a pen. Part A is done without a computer.

All three new programs go inside the existing day1_lab folder, so they can reuse config.py, tools.py and agent.py. Do not create a new virtual environment.

5. Video Tutorials

Watch Videos 1 and 2 before the lab if you have not already seen them.

No.	Title and Link	Use in this lab
1	ReAct AI Agents, clearly explained! https://www.youtube.com/watch?v=vFdlrZyKEwQ	Part A: the Thought, Action, Observation cycle you will trace on paper.
2	Chain-of-Thought (CoT) Prompting Explained https://www.youtube.com/watch?v=4aZFbuOUwmA	Parts C and D: why step-by-step prompting improves answers.
3	AI Agentic Design Patterns: ReAct Explained https://www.youtube.com/watch?v=WBgl9ce_zwM	Optional. ReAct presented as a reusable design pattern.
4	Chain of Thought (CoT) Prompts Explained with Examples https://www.youtube.com/watch?v=LdFB4J9SD9E	Optional. More CoT examples, with no coding.

If a link does not open, search the exact video title on YouTube.

6. Problem Statement

Part A and B use this multi-step question, with the same fee data as Day 1 (CS101 = Rs. 12,000, AI202 = Rs. 18,000, DS303 = Rs. 15,000):

Which is cheaper: CS101 and AI202 with a 10% scholarship, or all three courses with a 25% scholarship? And by how much?

Correct answer (work it out before you start, so you can judge the agent):

Option	Calculation
CS101 + AI202 with 10% off	$(12,000 + 18,000) \times 0.9 = \text{Rs. } 27,000$
All three with 25% off	$(12,000 + 18,000 + 15,000) \times 0.75 = \text{Rs. } 33,750$
Answer	The first option is cheaper, by Rs. 6,750

Parts C and D use three reasoning questions that need no tools, only careful thinking:

Question	Correct answer
Three courses cost Rs. 12,000, Rs. 18,000 and Rs. 15,000. With a 15% scholarship on the total, paid in 4 equal instalments, how much is each instalment?	$45,000 \times 0.85 = 38,250; \div 4 = \text{Rs. } 9,562.50$
A lab has 18 computers, shared by 2 students each morning and 3 each afternoon. How many student sittings in one day?	$(18 \times 2) + (18 \times 3) = 90$
Ravi is taller than Kumar. Kumar is taller than Arun. Priya is shorter than Arun. Who is tallest and who is shortest?	Tallest: Ravi. Shortest: Priya.

7. Procedure: Part A – Trace ReAct on Paper

Close your laptop for this part. You are going to act as the agent.

Rules of the game

1. You may use only two tools: **get_course_fee(course_code)** and **calculator(expression)**.
2. You do not know any fee until a tool tells you. Write the observation only when you 'call' the tool.
3. Every row must be one of: Thought (what you need next and why), Action (one tool call), or Observation (the result).
4. One action per row. No arithmetic in your head: use the calculator tool.
5. Stop when you can state the final answer, and write it in the last row.

7.1 Worksheet

Fill this table for the question in Section 6. Add rows if you need more.

Step	Type	Content	Why this step
1	Thought		
2	Action		
3	Observation		
4	Thought		
5	Action		
6	Observation		
7	Thought		
8	Action		
9	Observation		
10	Thought		
11	Action		
12	Observation		
13	Final Answer		

Questions to answer below your trace: How many tool calls did you need? How many LLM calls would that be in real life? Could any two actions have been done at the same time (in parallel)?

A complete trace usually needs three fee lookups and two or three calculator calls. If you finished in two steps, check whether you did arithmetic in your head, which a real agent cannot do reliably.

8. Procedure: Part B – Compare With the Real Agent

Now open VS Code, go to the **day1_lab** folder, activate the environment, and create this file.

File: react_trace.py

```
"""Day 2, Part D: print the agent's real ReAct trace to compare with your paper trace."""
from agent import agent

QUESTION = ("Which is cheaper: CS101 and AI202 with a 10% scholarship, "
           "or all three courses with a 25% scholarship? By how much?")

print("QUESTION:", QUESTION, "\n")
print("--- the agent's actions and observations ---")
answer = agent(QUESTION, max_steps=8)
print("\nFINAL ANSWER:", answer)
```

Run it: `python react_trace.py`

Sample output (tool order and wording vary between runs and models):

```
QUESTION: Which is cheaper: CS101 and AI202 with a 10% scholarship, or all
three courses with a 25% scholarship? By how much?

--- the agent's actions and observations ---
step 1: get_course_fee({'course_code': 'CS101'}) -> 12000
step 1: get_course_fee({'course_code': 'AI202'}) -> 18000
step 1: get_course_fee({'course_code': 'DS303'}) -> 15000
step 2: calculator({'expression': '(12000 + 18000) * 0.9'}) -> 27000.0
step 3: calculator({'expression': '(12000 + 18000 + 15000) * 0.75'}) -> 33750.0
step 4: calculator({'expression': '33750 - 27000'}) -> 6750.0

FINAL ANSWER: Taking CS101 and AI202 with the 10% scholarship costs
Rs. 27,000, which is Rs. 6,750 cheaper than all three courses at Rs. 33,750.
```

Now **compare** the machine trace with your paper trace and fill in Section 11.1:

- Did the agent call the same tools you did?
- Did it call them in the same order?
- Did it use more steps or fewer steps than you?
- Did it call any tools in parallel (two calls printed under the same step number)?
- Did it reach the same final answer, Rs. 6,750?

If your model gives a wrong final answer, keep the trace. Look at the first observation that is wrong or missing: that is where the reasoning broke. Finding this point is the skill this lab is teaching.

9. Procedure: Part C – With and Without Chain-of-Thought

This part uses no tools at all. It measures what step-by-step prompting alone can do.

File: cot_compare.py

```

"""Day 2, Part B: the same question asked WITHOUT and WITH Chain-of-Thought."""
from config import client, MODEL, banner

QUESTIONS = [
    # 1. Multi-step arithmetic
    "A student takes three courses costing Rs. 12,000, Rs. 18,000 and Rs. 15,000. "
    "She gets a 15% scholarship on the total and pays the rest in 4 equal instalments. "
    "How much is each instalment?",
    # 2. Counting in two parts
    "A lab has 18 computers. In the morning each computer is shared by 2 students, "
    "and in the afternoon by 3 students. How many student sittings happen in one day?",
    # 3. Ordering / logic
    "Ravi is taller than Kumar. Kumar is taller than Arun. Priya is shorter than Arun. "
    "Who is the tallest and who is the shortest?",
]

DIRECT_PROMPT = "You are a helpful assistant. Give only the final answer. Do not explain."

COT_PROMPT = ("You are a helpful assistant. Solve the problem step by step. "
              "Number each step and show the calculation in that step. "
              "After the steps, write the last line exactly as: Final Answer: <answer>")

def ask(system_prompt, question):
    response = client.chat.completions.create(
        model=MODEL,
        messages=[{"role": "system", "content": system_prompt},
                  {"role": "user", "content": question}],
        temperature=0,
    )
    return response.choices[0].message.content.strip()

if __name__ == "__main__":
    banner("CHAIN-OF-THOUGHT COMPARISON")
    for number, question in enumerate(QUESTIONS, start=1):
        print("=" * 72)
        print(f"QUESTION {number}: {question}\n")
        print("--- WITHOUT CoT ---")
        print(ask(DIRECT_PROMPT, question), "\n")
        print("--- WITH CoT ---")
        print(ask(COT_PROMPT, question), "\n")

```

Run it: python cot_compare.py

Sample output for Question 1 (your model's wording will differ):

```

QUESTION 1: A student takes three courses costing Rs. 12,000, Rs. 18,000 and
Rs. 15,000 ...

--- WITHOUT CoT ---
Rs. 11,250                                     <-- WRONG (forgot the discount)

--- WITH CoT ---
Step 1: Total = 12000 + 18000 + 15000 = 45000
Step 2: Scholarship = 15% of 45000 = 6750
Step 3: Payable = 45000 - 6750 = 38250
Step 4: Each instalment = 38250 / 4 = 9562.5
Final Answer: Rs. 9,562.50                       <-- CORRECT

```

What to record for each of the three questions: was the direct answer correct, was the CoT answer correct, and how much longer was the CoT reply?

Observation	What it tells you
CoT fixes some wrong answers	The model had the ability, but not the space to use it. Writing the steps gives it that space.

Observation	What it tells you
Small models improve the most	A 1.5B model without CoT guesses; with CoT it often reaches the right result.
CoT replies are longer and slower	More tokens means more time and more cost. Accuracy is being bought, not given free.
CoT cannot invent facts	Notice that these three questions contain all the data needed. Ask a fee question instead and CoT will still hallucinate, because no tool is involved.

Extra experiment: add a fourth question to the list that requires private data, for example "What is the fee for AI202?", and run again. The CoT version will reason step by step over numbers it invented. This is the exact gap that ReAct fills.

10. Procedure: Part D – Self-Consistency

One reasoning path can go wrong. Self-consistency runs the same CoT prompt several times at a higher temperature and takes the answer that appears most often.

File: `self_consistency.py`

```

"""Day 2, Part C: run the same CoT prompt several times and take the majority answer."""
from collections import Counter
from config import client, MODEL, banner
from cot_compare import COT_PROMPT, QUESTIONS

RUNS = 5
TEMPERATURE = 0.8          # deliberately NOT 0, so each run can differ

def final_answer(text):
    """Pull out the text after 'Final Answer:' (the last line of a CoT reply)."""
    for line in reversed(text.splitlines()):
        if "final answer" in line.lower():
            return line.split(":", 1)[-1].strip()
    return text.splitlines()[-1].strip() if text.strip() else "(empty)"

def run_many(question, runs=RUNS, temperature=TEMPERATURE):
    answers = []
    for attempt in range(1, runs + 1):
        response = client.chat.completions.create(
            model=MODEL,
            messages=[{"role": "system", "content": COT_PROMPT},
                     {"role": "user", "content": question}],
            temperature=temperature,
        )
        answer = final_answer(response.choices[0].message.content)
        print(f"  run {attempt}: {answer}")
        answers.append(answer)
    return answers

if __name__ == "__main__":
    banner("SELF-CONSISTENCY")
    question = QUESTIONS[0]
    print("QUESTION:", question, "\n")
    answers = run_many(question)
    winner, count = Counter(answers).most_common(1)[0]
    print(f"\nMajority answer ({count} of {len(answers)} runs): {winner}")

```

Run it: `python self_consistency.py`

Sample output:

```

run 1: Rs. 9,562.50
run 2: Rs. 9,562.50
run 3: Rs. 11,250
run 4: Rs. 9,562.50
run 5: Rs. 9,562.50

```

Majority answer (4 of 5 runs): Rs. 9,562.50

Parameter	Why this value
temperature = 0.8	Deliberately not 0. Randomness is needed so that the runs explore different reasoning paths. At 0 all five runs would be nearly identical and voting would be pointless.
RUNS = 5	An odd number avoids ties, and five runs are enough to show the effect without a long wait. Research uses far more.
Voting on the final answer only	The reasoning may be worded differently every time, so only the last line is compared.

Try this: change TEMPERATURE to 0 and run again. The five answers become almost identical, so voting adds nothing. Then set it to 1.2 and see whether the answers become too scattered to agree.

11. Observations

11.1 Paper trace vs agent trace

Item	Your paper trace	The agent
Number of fee lookups		
Number of calculator calls		
Total steps		
Any tools called in parallel? (Y/N)		
Final answer		
Correct? (Y/N)		

11.2 Chain-of-Thought comparison

Question	Without CoT correct? (Y/N)	With CoT correct? (Y/N)	Which reply was longer?
Q1 instalments			
Q2 lab sittings			
Q3 tallest and shortest			

11.3 Self-consistency

Item	Value
Answers seen across the 5 runs	
Majority answer	

Item	Value
Was the majority answer correct?	
Result when temperature = 0	

12. Discussion Questions

1. Your paper trace and the agent's trace probably differ. Does a different order of steps make either one wrong?
2. In Part C, the model improved when asked to think step by step. If the ability was already there, why did it not do this by itself?
3. Chain-of-Thought could not answer the fee question correctly. Which component from Day 2 theory is missing, and which pattern supplies it?
4. Self-consistency needs a non-zero temperature, but Day 1 used temperature 0 for tool calling. Explain why the two settings differ.
5. The agent solved the Section 6 question in around six tool calls. How would Plan-and-Execute handle the same question, and how many LLM calls would it need?

13. Troubleshooting

Problem	Solution
ModuleNotFoundError: No module named 'config'	The new files must be inside the day1_lab folder, beside config.py. Check the file location in the VS Code explorer.
The agent answers without calling any tool	The model ignored the tool policy. Re-run, or switch to Groq or Hugging Face in .env. Record it as a failure mode.
Stopped: maximum steps reached	This question needs up to six tool calls; react_trace.py already passes max_steps=8. Raise it further only if your model is taking extra steps.
The CoT reply has no 'Final Answer:' line	Small models sometimes ignore the format instruction. self_consistency.py falls back to the last line, but the votes become noisy. Re-run, or make the instruction firmer.
All five self-consistency runs are identical	TEMPERATURE was left at 0, or the model ignores it. Set it to 0.8 or higher.
Rate limit (429) on Groq or Hugging Face	self_consistency.py makes five calls quickly. Wait a minute, reduce RUNS to 3, or switch to Ollama.
Answers are wrong even with CoT	Expected on a 1.5B model for the instalment question. Note it, then try a larger model and compare. Model size is part of the result.

14. Viva Questions

1. What are the three row types in a ReAct trace, and what does each contain?
2. Why can a thought not change anything in the outside world?
3. What is the difference between Chain-of-Thought and ReAct?
4. Why did CoT fail on the course-fee question?
5. Why does self-consistency need temperature above 0?
6. Why is voting done on the final answer rather than on the whole reply?

7. What does it mean when two tool calls are printed under the same step number?
8. Name one advantage and one disadvantage of Chain-of-Thought prompting.

15. Result

Thus, a ReAct run was traced on paper and compared with the trace produced by a working agent, and the effect of Chain-of-Thought prompting and self-consistency was measured on a set of reasoning questions. The observations show that
